

Bàn sơ về ứng dụng của thuật toán không hoàn hảo trong thi tin học

Hu Weidong

Trường Trung học Changjun, Trường Sa, Hồ Nam

2005

Nguồn gốc: Bàn sơ về ứng dụng của thuật toán không hoàn hảo trong thi tin học

IOI China candidate team papers / 2005 / Hu Weidong.doc

Tóm tắt

Thuật toán không hoàn hảo là thuật toán chấp nhận mất một lượng nhỏ tính đúng đắn để đổi lấy hiệu quả thời gian, bộ nhớ hoặc độ phức tạp lập trình. Bài viết giới thiệu vài phương pháp quan trọng của loại thuật toán này, cùng ưu điểm và nhược điểm của chúng. Qua đó có thể thấy: không phải thuật toán hoàn toàn đúng lúc nào cũng tốt hơn thuật toán không hoàn hảo; chính sự không hoàn toàn ấy đôi khi giúp một thuật toán “không đúng tuyệt đối” thể hiện tốt hơn thuật toán đúng ở nhiều mặt.

Từ khóa: thuật toán không hoàn hảo; tham lam ngẫu nhiên; kiểm tra bằng lấy mẫu; bỏ qua cục bộ.

1 Dẫn nhập

Thông thường, khi nghiên cứu thuật toán, ta gần như luôn theo đuổi thuật toán hoàn toàn đúng. Nhưng trong ứng dụng thực tế, rất nhiều kỹ thuật không xử lý dữ liệu với tính đúng đắn tuyệt đối. Nén ảnh, âm thanh, video với tỉ lệ nén cao thường là nén mất mát; nhiều phương pháp kiểm chứng mật khẩu dùng ánh xạ nhiều-một, nên vượt qua kiểm chứng không đồng nghĩa với việc chắc chắn là mật khẩu gốc; công cụ tìm kiếm không thể đảm bảo tìm được mọi kết quả thỏa mãn. Dù vậy, ta vẫn dùng chúng, vì chúng có ý nghĩa thực tế: nén mất mát giúp tệp nhỏ hơn rất nhiều; ánh xạ nhiều-một vẫn khó bị đảo ngược; công cụ tìm kiếm tuy không đủ 100% nhưng nhanh và tiện lợi.

Vậy trong thi tin học, có thể dùng thuật toán không hoàn hảo để giải bài hay không? Bài viết này thử giới thiệu một số thuật toán không hoàn hảo thường gặp, với hy vọng gợi mở thêm cách nghĩ cho người đọc.

Trước hết, cần chấp nhận một điểm: trong tin học, thậm chí trong toàn bộ khoa học máy tính, thuật toán tuyệt đối đúng không nhất thiết là thuật toán tốt nhất. Một thuật toán đúng trong hầu hết trường hợp có thể có triển vọng hơn một thuật toán hoàn toàn đúng, vì nó dùng ít thời gian hoặc bộ nhớ hơn, dễ cài đặt hơn, hoặc không đòi hỏi kiến thức quá sâu để hiểu và áp dụng.

2 Một số phương pháp cơ bản

2.1 Tham lam ngẫu nhiên

Tham lam ngẫu nhiên là một loại thuật toán không hoàn hảo được dùng khá rộng, đặc biệt trong các bài yêu cầu nghiệm tốt chứ không nhất thiết để tìm nghiệm tối ưu bằng cách chắc chắn.

Nó kết hợp ngẫu nhiên với tham lam: ở mỗi bước, ta cố gắng chọn quyết định tốt, nhưng không nhất thiết là quyết định tốt nhất tuyệt đối. Bằng cách chạy nhiều lần, thuật toán thường thu được một nghiệm khá tốt. Đôi khi, do khó đánh giá chất lượng quyết định, chỉ cần ngẫu nhiên nhiều lần cũng có thể cho kết quả tốt.

2.1.1 Ví dụ: khống chế dịch bệnh

Mô tả ngắn. Cho một cây truyền bệnh, trong đó gốc là đỉnh đã nhiễm. Ở mỗi thời điểm, các đỉnh đã nhiễm truyền bệnh cho con của chúng. Nếu tại thời điểm t cắt cạnh giữa đỉnh A và cha B của nó, thì từ sau thời điểm t , bệnh không truyền từ B sang A . Mỗi thời điểm chỉ cắt được một đường truyền. Hỏi nên cắt như thế nào để số người bị nhiễm là ít nhất.

Bản gốc ghi rằng thuật toán chuẩn là tìm kiếm; lời giải chi tiết được đặt ở phụ lục riêng về bài khống chế dịch bệnh.

Phân tích. Nếu một đỉnh đã bị nhiễm, về sau cắt cạnh nối nó với cha không còn ý nghĩa. Nếu tại một thời điểm, cắt cạnh tới A là hiệu quả, cắt cạnh tới B cũng hiệu quả, và B là hậu duệ của A , thì cắt cạnh tới A tốt hơn cắt cạnh tới B . Hơn nữa, ở thời điểm t , các đỉnh có thể bị nhiễm chỉ nằm trong t tầng đầu, nên cạnh cần cắt tự nhiên nằm giữa tầng t và tầng $t + 1$.

Một chiến lược kinh nghiệm là mỗi lần cắt khỏi cây con có nhiều đỉnh nhất. Nhưng chiến lược này không thích hợp cho mọi trường hợp; đôi khi chọn cây con lớn thứ hai hoặc thứ ba lại tốt hơn cho kết quả cuối. Vì vậy có thể chạy tham lam nhiều lần, mỗi lần chọn ngẫu nhiên trong số các phương án có khả năng cắt được nhiều đỉnh, hoặc gán xác suất cao hơn cho phương án cắt được nhiều đỉnh. Dù nhiều lần chạy cho kết quả kém hơn tham lam đơn thuần, khi số lần ngẫu nhiên đủ lớn, xác suất xuất hiện đáp án đúng hoặc rất tốt sẽ tăng lên.

Với bài này, thậm chí có thể đặt xác suất chọn mọi phương án như nhau, tức là gần như dùng ngẫu nhiên thuần túy, và vẫn có thể đạt điểm tối đa trên dữ liệu gốc.

Tiểu kết. Tham lam ngẫu nhiên là một công cụ quan trọng trong thi tin học. Vì thường dựa trên một tham lam đơn giản, nó có độ phức tạp cài đặt thấp và thời gian chạy nhỏ. Chạy nhiều lần là thủ đoạn then chốt; nhờ đó ta thường thu được nghiệm rất tốt.

2.2 Kiểm tra bằng lấy mẫu

Lấy mẫu nghĩa là chọn một phần đơn vị từ tổng thể, rồi dùng kết quả của mẫu để suy ra chỉ tiêu của tổng thể. Trong một số bài, đề cho một tập các đối tượng cần kiểm tra S . Nếu tồn tại một đối tượng thỏa điều kiện A , ta nói S có tính chất P ; nếu không có đối tượng nào thỏa A , ta nói S không có tính chất P .

Muốn quyết định chính xác S có tính chất P hay không, về nguyên tắc phải liệt kê toàn bộ đối tượng trong S . Nếu còn một đối tượng chưa kiểm tra và mọi đối tượng đã kiểm tra đều không thỏa A , ta vẫn chưa thể khẳng định các đối tượng còn lại không thỏa A .

Nhưng nếu tổng thể có tính chất: hoặc không có đối tượng thỏa A , hoặc số đối tượng thỏa A khá nhiều, thì chỉ cần chọn một số ít mẫu đại diện để kiểm tra cũng có thể đảm bảo kết quả gần đúng. “Đại diện” ở đây có nghĩa là lấy cả những mẫu đặc biệt lẫn những mẫu bình thường, giống như khảo sát xã hội cần lấy người từ nhiều nhóm nghề nghiệp và độ tuổi khác nhau.

Ví dụ, giả sử tổng thể S có 10000 đối tượng, trong đó 100 đối tượng thỏa A . Nếu lấy ngẫu nhiên 100 mẫu, xác suất phát hiện S có tính chất P vào khoảng 63.6%; lấy 200 mẫu thì khoảng 86.9%; lấy 300 mẫu thì khoảng 95.3%. Chỉ khoảng 70 mẫu đã đạt xác suất gần 50%, còn khoảng 230 mẫu đạt gần 90%.

Một ví dụ khác: nếu S là mọi tập con của bảng chữ cái A tới Z, còn điều kiện A là tập con chứa đồng thời A tới G, thì ngoài mẫu ngẫu nhiên, ta nên kiểm tra một số tập đặc biệt như tập rỗng, tập đầy đủ, các tập một phần tử, v.v. Trong ví dụ này, tập đầy đủ hiển nhiên thỏa điều kiện.

2.2.1 Ví dụ: Logic trực giác

Mô tả ngắn. Cho đồ thị có hướng không chu trình $G = (V, E)$. Trên V định nghĩa quan hệ thứ tự bộ phận:

$$x \leq y \iff \text{tồn tại đường đi từ } x \text{ tới } y.$$

Với tập đỉnh S , định nghĩa $\text{Max}(S)$ là tập các phần tử cực đại của S :

$$\text{Max}(S) = \{x \in S \mid \nexists y \in S, x \neq y, x \leq y\}.$$

Gọi

$$\beta = \{S \mid \text{Max}(S) = S\}.$$

Đặt $0 = \text{Max}(V)$ và $1 = \emptyset$, khi đó $0, 1 \in \beta$. Trên các phần tử của β định nghĩa các phép toán:

$$P \Rightarrow Q = \{x \in Q \mid \nexists y \in P, x \leq y\},$$

$$P \vee Q = \text{Max}(P \cup Q),$$

$$P \wedge Q = \text{Max}(\{x \in V \mid \exists y \in P, \exists z \in Q, x \leq y, x \leq z\}),$$

$$\neg P = (P \Rightarrow 0), \quad P \equiv Q = ((P \Rightarrow Q) \wedge (Q \Rightarrow P)).$$

Bài toán hỏi: với một biểu thức có biến, liệu với mọi cách gán biến bằng phần tử của β , kết quả có luôn bằng 1 hay không?

Giới hạn gồm $|V| \leq 100$, $|E| \leq 5000$, mỗi đồ thị có tối đa 20 biểu thức, mỗi biểu thức dài không quá 254, và $|\beta| \leq 100$.

Phân tích. Giới hạn về $|\beta|$ gợi ý rằng có thể liệt kê mọi cách gán biến. Nếu muốn hoàn toàn đúng, số cách gán biến về cơ bản không thể giảm, có thể đạt tới 10^6 . Khi đó cần giảm chi phí tính biểu thức. Vì các phép cơ bản có thể mất $\mathcal{O}(|V|)$, $\mathcal{O}(|E|)$ hoặc hơn, còn một biểu thức có thể dùng hàng trăm phép cơ bản, liệt kê trực tiếp khó đáp ứng thời gian.

Do $|\beta| \leq 100$, ta có thể đánh số từng phần tử của β bằng một số nguyên, rồi tiền xử lý bảng kết quả cho các phép toán cơ bản giữa các số ấy. Khi tính biểu thức, mỗi phép chỉ còn là tra bảng $\mathcal{O}(1)$.

Một cách khác là lấy mẫu. Ta xem các giá trị đặc biệt của biến như 0, 1, ... là mẫu đặc biệt, các giá trị còn lại là mẫu thường. Chọn một số mẫu đặc biệt và một số mẫu thường để kiểm tra. Nếu với một biểu thức, mọi mẫu đều cho kết quả đúng, ta chấp nhận rằng biểu thức đúng; nếu có mẫu sai, biểu thức sai. Kinh nghiệm cho thấy lấy dưới 1000 mẫu đã có thể đạt xác suất đúng khá cao.

Với bài nhiều bộ kiểm tra, cần chú ý rằng xác suất đúng của từng nhóm phải rất cao. Nếu một điểm có 20 nhóm dữ liệu và mỗi nhóm đúng 95%, xác suất cả điểm đúng chỉ là 0.95^{20} , chưa tới 36%. Nếu mỗi nhóm đúng 99%, toàn điểm khoảng 82%; nếu mỗi nhóm đúng 99.9%, toàn điểm mới khoảng 98%. Vì vậy khi dùng lấy mẫu, số mẫu nên càng nhiều càng tốt trong giới hạn thời gian.

2.2.2 Ứng dụng trong kiểm tra số nguyên tố

Lấy mẫu rất hữu ích trong kiểm tra số nguyên tố. Xét phương pháp Miller–Rabin theo cách diễn đạt trong bản gốc. Với số cần kiểm tra n , viết

$$n - 1 = d \cdot 2^s, \quad d \text{ lẻ.}$$

Với một cơ sở a , nếu

$$a^d \equiv 1 \pmod{n}$$

hoặc tồn tại $0 \leq r < s$ sao cho

$$a^{d2^r} \equiv -1 \pmod{n},$$

thì n vượt qua kiểm tra mạnh với cơ sở a . Có thể dùng nhân đôi để kiểm tra điều này trong $\mathcal{O}(\log n)$.

Với hợp số c , khi chọn cơ sở nhỏ hơn c , xác suất để c vượt qua kiểm tra mạnh không quá $1/4$ theo kết quả của Monier và Rabin. Vì vậy, nếu chọn ngẫu nhiên k cơ sở nhỏ hơn n , xác suất đúng ít nhất là

$$1 - 4^{-k},$$

một xác suất rất tốt; thường chỉ cần vài chục cơ sở.

Nếu không chọn ngẫu nhiên mà kiểm tra một số cơ sở đặc biệt là các số nguyên tố nhỏ nhất, kết quả cũng rất mạnh. Chỉ dùng cơ sở 2 thì phản ví dụ nhỏ nhất là 2047; dùng 2, 3 thì phản ví dụ nhỏ nhất vượt 10^6 ; dùng 2, 3, 5 thì vượt $2 \cdot 10^7$; dùng 2, 3, 5, 7 thì vượt $3 \cdot 10^9$, lớn hơn giới hạn số nguyên có dấu 32 bit. Do đó trong nhiều ngữ cảnh, kiểm tra vài cơ sở cố định như 2, 3, 5, 7 đã đủ đảm bảo.

So với phép thử chia từ 2 tới $\sqrt{n}$ có độ phức tạp $\mathcal{O}(\sqrt{n})$, cách kiểm tra bằng lấy mẫu chỉ cần $\mathcal{O}(\log n)$ cho mỗi cơ sở và số cơ sở rất ít, nên tổng thể hiệu quả hơn rõ rệt.

Tiểu kết. Vì lấy mẫu chỉ kiểm tra một phần tổng thể, số lần kiểm tra ít hơn rất nhiều so với liệt kê đầy đủ, nhờ đó thời gian chạy giảm mạnh.

2.3 Phương pháp bỏ qua cục bộ

Trong tin học, có những bài toán có phân loại trường hợp rất phức tạp, và một số trường hợp xử lý cực kỳ rắc rối nhưng không phải tình huống chính: chúng xuất hiện với xác suất nhỏ hoặc ảnh hưởng rất ít tới đáp án. Đôi khi bỏ qua những trường hợp phức tạp nhưng ít ảnh hưởng ấy lại cho kết quả hài lòng.

2.3.1 Ví dụ: Polygon

Mô tả ngắn. Trong bài này, mọi đa giác đều là đa giác lồi. Cho hai đa giác A và B , tổng Minkowski của chúng là đa giác chứa mọi điểm dạng

$$(x_1 + x_2, y_1 + y_2),$$

trong đó $(x_1, y_1) \in A$ và $(x_2, y_2) \in B$.

Bài toán xét một phép ngược của tổng Minkowski: cho đa giác P , tìm hai đa giác A, B sao cho P là tổng Minkowski của A và B ; A có từ 2 tới 4 đỉnh; và số đỉnh của A càng nhiều càng tốt.

Lời giải chính thức có độ phức tạp $\mathcal{O}(N^2 m^2)$, với N là số đỉnh của P và m là số điểm nguyên nhiều nhất trên một cạnh của P .

Phân tích. Nếu chỉ xét hình dạng và kích thước, phép cộng Minkowski của hai đa giác có thể hiểu như sau: xem các cạnh của hai đa giác là các vector có thứ tự, sắp tất cả vector theo góc cực, nối chúng liên tiếp rồi gộp các vector cùng hướng và cùng phương.

Từ quá trình cộng này, mỗi cạnh của A hoặc B sẽ tương ứng với một cạnh của P cùng phương cùng hướng, và độ dài trong P không nhỏ hơn độ dài gốc. Vì vậy, để tách P thành tổng của một đa giác A có K cạnh và một đa giác B , ta cần tìm K cạnh trong P , lấy một đoạn trên mỗi cạnh sao cho K đoạn ấy khép lại thành một đa giác. Đa giác đó là A , còn phần còn lại nối đuôi nhau thành B .

Tìm một đa giác hai cạnh chỉ cần một cặp vector cùng phương ngược chiều phù hợp. Tìm tam giác có thể liệt kê ba cạnh và kiểm tra, độ phức tạp khoảng $\mathcal{O}(N^3m^3)$, đã khá tốt với bài dạng output-only. Nhưng tìm tứ giác bằng cách liệt kê bốn cạnh có thể lên tới $\mathcal{O}(N^4m^4)$. Có thể liệt kê ba cạnh rồi tìm cạnh thứ tư bằng nhị phân, nhưng cạnh thứ tư có thể chỉ là một đoạn của một cạnh trong P , số đoạn có thể là Nm , khó cấp phát và lưu trữ; hơn nữa còn phải lưu đoạn thuộc cạnh gốc nào và chiếm tỉ lệ bao nhiêu, cài đặt dễ sai.

Lúc này có thể đưa ra giả thiết táo bạo: cạnh thứ tư là một cạnh đầy đủ của P . Khi đó chỉ cần lưu các cạnh gốc của P , mọi đoạn xét đều là cạnh hoàn chỉnh, số tình huống cài đặt giảm mạnh. Với dữ liệu chính thức, cách bỏ qua cục bộ này giải được 9 điểm kiểm tra, và các nghiệm tìm được đều là tối ưu; điểm còn lại có thể xử lý bằng tay.

2.3.2 Ứng dụng trong kiểm tra điểm nằm trong đa giác

Xét bài toán xác định điểm P có nằm trong một đa giác đơn hay không. Giả sử đã xử lý riêng trường hợp P nằm trên cạnh. Cách kinh điển là bắn một tia từ P và đếm số giao điểm với đa giác: lẻ thì ở trong, chẵn thì ở ngoài.

Khó khăn là tia có thể đi qua một đỉnh của đa giác, thậm chí trùng với một cạnh. Khi đó cần xử lý trường hợp đặc biệt cẩn thận, vì các cấu hình rất gần nhau có thể cho quy tắc đếm khác nhau.

Theo phương pháp bỏ qua cục bộ, ta chọn một tia “rất bình thường”: lấy ngẫu nhiên một điểm Q khác P trên mặt phẳng, với tọa độ có phần thập phân, rồi bắn tia từ P qua Q . Xác suất tia đi đúng qua đỉnh hoặc trùng cạnh là cực nhỏ. Nếu muốn chắc hơn, lấy vài điểm Q khác nhau, kiểm tra nhiều lần rồi chọn kết quả xuất hiện nhiều nhất.

Tiểu kết. Bỏ qua một số trường hợp phức tạp có thể làm giảm cả độ phức tạp suy nghĩ lẫn độ phức tạp lập trình. Dù có thể tạo ra sai số nhất định, sai số ấy thường nhỏ trong bối cảnh phù hợp. Vì vậy, đôi khi đây vẫn là một lựa chọn rất tốt.

3 Cơ sở: lớp RP và thuật toán Monte Carlo

Những phương pháp trên dường như dựa vào “may mắn”, nhưng chúng có cơ sở lý thuyết. Nếu một bài toán có một thuật toán ngẫu nhiên cho kết quả mong muốn với xác suất lớn hơn 50%, bài toán đó thuộc lớp RP; thuật toán tương ứng được gọi là thuật toán Monte Carlo.

Nếu một bài toán thuộc lớp RP, ta có thể chạy thuật toán Monte Carlo nhiều lần để thu được một thuật toán “gần như lần nào cũng đúng”. Vì RP và Monte Carlo không phải trọng tâm của bài viết, bản gốc chỉ nêu ngắn gọn và khuyến khích người đọc tự tham khảo thêm.

4 Điểm chung của thuật toán không hoàn hảo

Thuật toán không hoàn hảo có nhiều dạng, nhưng chúng không hoàn toàn tách rời. Điểm chung của chúng là tính không đầy đủ. Chính sự không đầy đủ ấy đem lại những ưu thế mà thuật toán hoàn toàn đúng thường không có, đồng thời cũng dẫn tới việc thuật toán không đảm bảo đúng tuyệt đối.

5 Ưu điểm và nhược điểm

5.1 Ưu điểm

1. **Tiêu hao thời gian và bộ nhớ thấp.** Đây là ưu điểm nổi bật nhất, và cũng là lý do chính để dùng thuật toán không hoàn hảo.
2. **Độ phức tạp lập trình thấp.** Thuật toán có thể né tránh nhiều xử lý rườm rà hoặc bỏ qua trường hợp rất khó xử lý. Trong thi đấu, cài đặt đơn giản đôi khi quan trọng không kém độ phức tạp lý thuyết thấp.
3. **Độ phức tạp suy nghĩ thấp.** Vì có thể bỏ qua những tình huống quá khó, quá trình thiết kế thuật toán cũng nhẹ hơn.
4. **Giảm lỗi lập trình.** Một thuật toán rất xuất sắc có thể thất bại vì một lỗi nhỏ trong cài đặt; ngược lại, một thuật toán không hoàn hảo nhưng dễ viết có thể tránh được lỗi và cho kết quả bất ngờ tốt.

5.2 Nhược điểm

Nhược điểm hiển nhiên là tính không đúng tuyệt đối. Giá trị của thuật toán không hoàn hảo nằm ở chính sự không đúng tuyệt đối này, nhưng nó cũng khiến kết quả phụ thuộc không chỉ vào thuật toán và mã nguồn, mà còn vào dữ liệu. Nếu dữ liệu yếu, thuật toán có thể đạt điểm cao; nếu dữ liệu mạnh, kết quả có thể không lý tưởng.

6 Kết luận

Bài viết giới thiệu một số phương pháp quan trọng của thuật toán không hoàn hảo. Qua các ví dụ, có thể thấy thuật toán đúng không nhất thiết luôn tốt hơn thuật toán không hoàn toàn đúng. Do tính không đầy đủ, thuật toán không hoàn hảo đôi khi lại vượt thuật toán đúng về thời gian, bộ nhớ, độ khó suy nghĩ và độ khó cài đặt. Sử dụng hợp lý loại thuật toán này có thể giúp ta thu được kết quả rất đáng hài lòng.

Lời cảm ơn

Xin chân thành cảm ơn thầy Xiang Qizhong đã hướng dẫn và giúp đỡ trong quá trình viết bài. Xin cảm ơn huấn luyện viên Liu Rujia đã chỉ dẫn và gợi mở. Xin cảm ơn Wang Jun, Ren Kai, Xiao Xiangning và Zhou Gelin đã hỗ trợ, đọc bài và góp ý quý báu; đặc biệt cảm ơn Zhou Gelin đã cung cấp lời giải bài *Không chế dịch bệnh*.

Tài liệu tham khảo

- Zhou Yongji, *Bàn về nguyên lý và thiết kế thuật toán ngẫu nhiên*, 1999.

- Eric W. Weisstein, *The CRC Concise Encyclopedia of Mathematics*, CRC Press.
- Zhou Peide, *Hình học tính toán: phân tích và thiết kế thuật toán*, Nhà xuất bản Đại học Thanh Hoa và Nhà xuất bản Khoa học Kỹ thuật Quảng Tây.
- Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Nhà xuất bản Đại học Thanh Hoa.
- Ioannis Emiris và Elias Tsigaridas, *IOI'04: Solution of Polygon*, 2004.

A Ghi chú về phụ lục

Bản gốc kèm bảng các strong pseudoprime nhỏ nhất theo tập cơ sở nguyên tố nhỏ, phát biểu gốc của bài NOIP 2003 *Khống chế dịch bệnh*, và phát biểu tiếng Trung của bài IOI 2004 *Polygon*. Trong kho bản dịch này, các tài liệu liên quan đã được xử lý riêng ở các tệp dịch tương ứng; vì vậy phụ lục ở đây chỉ ghi nhận vai trò của chúng, không lặp lại toàn bộ phát biểu dài.